TRD - Documento de Requisitos Técnicos

Módulo iFood: arquitetura, dados e integração

Versão 1.0 - 16/10/2026

Situação: rascunho para validação

1. Contexto técnico

O Let's Finance é um aplicativo React 18 + Vite + TypeScript (shadcn/ui) sobre Supabase (PostgreSQL, Auth, Edge Functions). O código é sincronizado com o Lovable, que grava direto no ramo **main** e regenera **types.ts**. O SQL de produção é aplicado manualmente no SQL Editor (arquivos em **scripts/producao/**, cópia idêntica das migrações em **supabase/migrations/**). O CI roda lint, testes e build.

Este documento detalha a implementação técnica do módulo iFood descrito no PRD.

2. Arquitetura

Camada	Componente	Responsabilidade
Fonte	API para parceiros do iFood	Pedidos (Merchant), valores financeiros (Financial), cardápio (Catalog)
Coleta	Edge Function **ifood-sync** (Deno)	Autentica por loja, busca pedidos e financeiro, grava de forma idempotente
Agendamento	pg_cron + pg_net (ou agendador do Supabase)	Dispara a coleta periodicamente e sob demanda
Dados	PostgreSQL (tabelas ifood_*)	Armazenamento, restrições de unicidade, RLS
Cálculo	Funções/visões SQL	Custo por pedido, por produto e conferência de repasses
Interface	Páginas React em /ifood	Lojas, pedidos, custos e repasses (hooks com RPC)

Fluxo: agendador -> ifood-sync (uma execução por loja ativa) -> autentica no iFood com os segredos da loja -> busca eventos/pedidos -> grava em ifood_orders/items/financials (upsert) -> atualiza last_sync_at ou last_error em ifood_stores.

3. Modelo de dados (implementado - SQL 13)

Migração **20261016120000_ifood_base.sql** e cópia **scripts/producao/13-ifood-base.sql** (idempotente). Verificada em Postgres descartável é coberta por teste vitest (script igual a migração, RLS e ausência de credenciais).

ifood_stores

Coluna	Tipo	Observação
id	uuid PK	gen_random_uuid()
name	text NOT NULL	Nome da loja
merchant_id	text UNIQUE	Identificador no iFood; opcional até ser informado
unit_id	uuid FK units	Unidade do sistema
secret_prefix	text	Prefixo dos segredos (ex.: IFOOD_LOJA1); o segredo não fica no banco
active	boolean	Padrão true
last_sync_at, last_error	timestamptz, text	Estado da última coleta
created_at, updated_at	timestamptz	Trigger update_updated_at

ifood_orders

Coluna	Tipo	Observação
id	uuid PK	
store_id	uuid FK ifood_stores	UNIQUE (store_id, ifood_order_id)
ifood_order_id, display_id	text	Identificadores do pedido

Coluna	Tipo	Observação
status, order_type, delivered_by	text	Situação, tipo (entrega/retirada) e quem entrega (iFood/loja)
ordered_at	timestamptz	Índice (store_id, ordered_at)
sub_total, delivery_fee, additional_fees, benefits_total, order_amount	numeric(14,2)	Totais do pedido
payment_method	text	
raw	jsonb	Resposta original do iFood, para conferência

ifood_order_items e ifood_order_financials

Tabela	Colunas principais
ifood_order_items	order_id FK (cascade), line_no (UNIQUE com order_id), external_code (índice), ean, name, quantity numeric(14,3), unit_price, options_price, total_price
ifood_order_financials	order_id UNIQUE FK (cascade), gross_value, commission, payment_fee, delivery_cost, merchant_promo, ifood_promo, net_value, expected_payout_date, raw jsonb

Tabelas previstas nas próximas fases

- **ifood_product_map**: equivalência entre o código externo do iFood (por loja) e o produto interno.
- **products_cost** (ou importação do PDV): custo unitário por produto, com vigência.
- **ifood_payouts**: repasses previstos por data, com vínculo opcional a linha do extrato conciliada.

4. Segurança

- RLS ligada nas quatro tabelas; uma política FOR ALL para authenticated exigindo has_role admin ou financeiro (mesmo padrão das tabelas de conciliação).
- A Edge Function usa a chave de serviço (service role), que ignora RLS; ela nunca é exposta ao navegador.
- Credenciais do iFood (clientId/clientSecret por loja) somente em segredos do Supabase, ex.: IFOOD_LOJA1_CLIENT_ID e IFOOD_LOJA1_CLIENT_SECRET. O banco guarda só o prefixo.
- A função de coleta valida o chamador: só executa por agendador interno ou usuário Admin/Financeiro autenticado.
- Token de acesso do iFood mantido apenas em memória durante a execução (ou cache curto), nunca gravado em tabela.
- Dados pessoais do cliente final não são copiados da resposta; o campo raw deve ser filtrado antes de gravar.

5. Integração com o iFood

Atenção: os endpoints e campos abaixo foram descritos de memória e precisam ser conferidos na documentação oficial (developer.ifood.com.br) e nos módulos liberados do app antes da implementação.

Necessidade	API	Uso previsto
Autenticação	OAuth do iFood (token por credencial)	Obter token por loja (ou por autorização) a cada execução; renovar ao expirar
Pedidos novos	Merchant API - polling de eventos + confirmação (acknowledgment)	Consultar eventos periodicamente, confirmar o recebimento e identificar pedidos novos ou alterados
Detalhe do pedido	Merchant API - detalhe por ID	Itens, totais, benefícios, pagamento, tipo de entrega

Necessidade	API	Uso previsto
Valores financeiros	Financial API - vendas, ajustes, eventos e conciliação	Comissão, taxa de pagamento, entrega, promoções e repasse por pedido; fonte do histórico
Cardápio	Catalog API	Código externo e dados do produto para a ligação com o PDV

Pontos de atenção

- O polling de eventos é voltado a tempo real; a obtenção de histórico tende a vir da API Financeira ou de relatórios do portal. Validar na documentação.
- Respeitar limites de requisição e frequência recomendada; tratar HTTP 429 com espera progressiva.
- Cada loja pode ter app/autorização próprio: a função recebe a loja como parâmetro e le os segredos pelo secret_prefix.
- Eventos podem chegar repetidos ou fora de ordem: a gravação é sempre por upsert com a chave (store_id, ifood_order_id).

6. Edge Function ifood-sync

Contrato

Item	Definição
Entrada	JSON opcional: { store_id?, from?, to? }. Sem parâmetros: todas as lojas ativas, janela padrão dos últimos dias.
Saida	JSON por loja: { store_id, orders_upserted, items_upserted, financials_upserted, error? }
Autorização	Cabeçalho do agendador (segredo compartilhado) ou JWT de Admin/Financeiro
Idempotência	upsert por (store_id, ifood_order_id); itens por (order_id, line_no); financeiro por order_id
Erros	Capturados por loja; gravam last_error e seguem para a próxima loja; sucesso limpa last_error e atualiza last_sync_at

Passos por loja

- Ler segredos pelo secret_prefix e obter token no iFood.
- Buscar eventos/pedidos da janela e, para cada pedido, o detalhe.
- Mapear para as colunas de ifood_orders e ifood_order_items (sem dados pessoais do cliente).
- Buscar valores financeiros da janela (Financial API) e mapear para ifood_order_financials.
- Gravar em lotes dentro de transações curtas; confirmar os eventos no iFood somente após gravar.
- Atualizar o estado da loja (last_sync_at / last_error).

Agendamento: execução a cada poucos minutos para pedidos novos (conforme limites do iFood) e uma execução diária de reconciliação dos valores financeiros dos últimos dias. Parâmetros ajustáveis sem republicar o app.

7. Cálculo de custos

Medida	Formula (por pedido)
custo_plataforma	commission + payment_fee + delivery_cost + merchant_promo
líquido	gross_value - custo_plataforma (conferir com net_value)
custo_itens	soma(quantity x custo unitário do produto ligado)

Medida	Formula (por pedido)
resultado	net_value - custo_itens
custo_plataforma_item	custo_plataforma x (total_price do item / soma dos total_price do pedido)

Implementação em funções SQL pequenas, uma por medida, no padrão ja adotado no projeto (LANGUAGE sql, sem ponto e vírgula interno, arquivos curtos), porque o SQL Editor do Supabase falha com funções grandes. Cada função é coberta por cenário SQL em scripts/testes-sql/ e, quando houver regra duplicada no front, por teste de paridade.

Itens sem produto ligado ou sem custo ficam fora do custo dos itens e são sinalizados (nunca contados como custo zero silenciosamente).

8. Interface (React)

- Rotas sob /ifood (hoje existe a página inicial em implantação, src/pages/Ifood.tsx e item de menu em AppSidebar).
- Hooks de dados chamando RPC via callRpc (src/lib/rpc.ts) e consultas as tabelas ifood_*; erros exibidos, nunca convertidos em zero (padrão do Dashboard).
- Filtros na URL (loja, período, status) com useSearchParams, como no Dashboard.
- Cadastro de lojas restrito a Admin; demais telas a Admin e Financeiro.
- Item de menu hoje visível a todos; restringir por perfil quando a coleta entrar em produção.

9. Testes e qualidade

Tipo	Cobertura
SQL em Postgres descartável	Cenários por tabela/funcao (scripts/testes-sql/), incluindo RLS por papel e idempotência
Vitest (CI)	Script de produção igual a migração; formato das funções; hooks com mocks do Supabase; páginas com jsdom
Edge Function	Funções puras de mapeamento (resposta do iFood -> linhas) testadas com respostas de exemplo; verificação de tipos Deno no CI, como a função kaikin
Aceite com dados reais	Comparar, por loja e período, a soma de net_value com os repasses recebidos no extrato

10. Observabilidade

- Estado por loja em ifood_stores (last_sync_at, last_error) exibido na tela de lojas.
- Logs da Edge Function com loja, janela, contagens e tempo; sem segredos nem dados pessoais.
- Alerta visual no app quando uma loja ficar sem coletar por mais de um intervalo definido.

11. Implantação

- Mudanças de banco: arquivo SQL numerado em scripts/producao/ aplicado manualmente, com a migração idêntica em supabase/migrations/.
- Edge Function: publicada pelo Lovable/Supabase; segredos cadastrados pelo responsável diretamente no Supabase.
- Cada etapa em um PR próprio, com CI verde antes e depois do merge; Publish no Lovable quando houver mudança de tela.

- Reversão: as tabelas novas são independentes; desativar a loja (active = false) interrompe a coleta sem perder dados.

12. Riscos técnicos

Risco	Mitigação
Diferenças entre a documentação do iFood e este desenho	Validar endpoints na documentação oficial antes da fase 2; manter o mapeamento isolado em funções puras
Limites de requisição ou indisponibilidade do iFood	Espera progressiva, nova tentativa e janelas de reprocessamento
Crescimento do campo raw (jsonb)	Guardar apenas o necessário; política de retenção se o volume crescer
Funções SQL grandes falharem no SQL Editor	Funções pequenas e arquivos curtos, com teste que protege o formato
Alterações do Lovable no mesmo código	Sincronizar com o main antes de cada PR e manter o CI como barreira

13. Estado atual e próximos passos

Item	Situação
Menu e página iFood (PR 18)	No main
Tabelas ifood_* e RLS (SQL 13, PR 19)	No main e aplicado no banco
Edge Function ifood-sync	Não iniciada - depende de merchantId, tipo de acesso e segredos
Telas de pedidos e custos	Não iniciadas
Custo dos insumos	Fonte a definir